Lab 01 - Bitcoin Core in Regtest

UTXO state, raw transactions and an independent Python parser

Course: Introduction to Blockchain Programming | Lab: 01 | Environment: Docker, Bitcoin Core 27.1, Python 3.12

This report summarises the results I obtained from my private Bitcoin regtest network. I used Bitcoin Core to create and confirm transactions, and I wrote a separate Python parser to inspect raw transaction bytes.

Part A	Mine 101 blocks and inspect wallet and global UTXO state.
Part B	Manually construct a transaction spending 50 BTC, sending 10 BTC and returning change.
Part C	Parse legacy and SegWit transactions and compare results with Bitcoin Core.
Part D	Omit change deliberately and demonstrate the resulting 40 BTC miner fee.

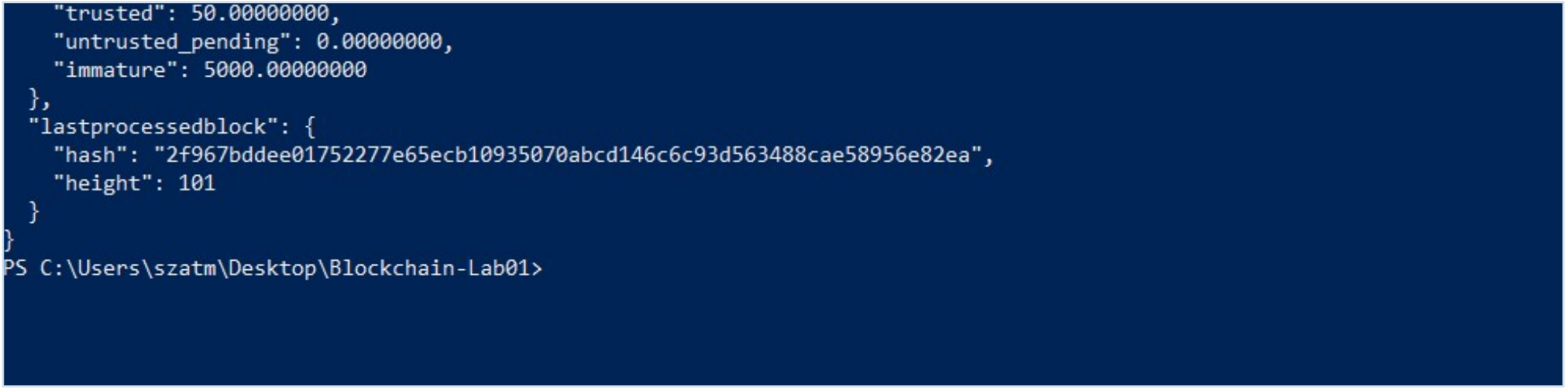
Repository: https://github.com/korneliasara/blockchain-lab01-bitcoin-parser

Evidence: screenshots were captured during my lab sessions. All BTC amounts are regtest coins, not real bitcoin. The screenshots show the commands and results I observed.

Part A - UTXO state after mining

Wallet evidence: 101 blocks, 50 BTC mature and 5,000 BTC immature

I created the lab01 wallet and mined 101 blocks. The getbalances output showed 50 BTC available and 5,000 BTC immature. The screenshot below records these values.



Part A - listunspent and the node UTXO set

Required evidence: a single mature output and 101 total unspent outputs

The wallet listed one spendable output: txid b64d9ac2...056fde38, vout 0, amount 50 BTC, 101 confirmations. gettxoutsetinfo reported height 101, txouts 101 and total_amount 5050 BTC.

The other 100 coinbase outputs were immature. Coinbase rewards need 100 more blocks built on top of their block before they can be spent. The node counts immature UTXOs; listunspent does not show them.

```
}
PS C:\Users\szatm\Desktop\Blockchain-Lab01> docker compose exec bitcoind bitcoin-cli -rpcwallet=lab01 listunspent
[
  {
    "txid": "b64d9ac2ec91ed2aa71738f2cdb689768b2f53d04f61d7eef3528b2d056fde38",
    "vout": 0,
    "address": "bcrt1qdddds2l0ug77l0k5tsy2q5p8x3f2ju6apprywuz",
    "label": "",
    "scriptPubKey": "00146b5b057dfc47bdf7da8b81140a04e68a552e6ba1",
    "amount": 50.00000000,
    "confirmations": 101,
    "spendable": true,
    "solvable": true,
    "desc": "wpkh([d206617d/84h/1h/0h/0/1]03c619ff767c4df9f5a16ccb368ae23a8bde75ff4a9a99cf20e102fd682b962b14)#6af6s44z",    "parent_descs": [
      "wpkh(tpubD6NzVbkrYhZ4XhU93AmhizGUUqsw6hPW5cQM8kmUYz3u6fdUit5dph6AL2pNsfC1pbxW1Poo6ZcnNjkS9Td5DWW5GYfqFe3qkHgyK7UW5X7/84h/1h/0h/0/*)#hjwqn34h"
    ],
    "safe": true
  }
]
PS C:\Users\szatm\Desktop\Blockchain-Lab01> docker compose exec bitcoind bitcoin-cli gettxoutsetinfo
{
  "height": 101,
  "bestblock": "2f967bddee01752277e65ecb10935070abcd146c6c93d563488cae58956e82ea",
  "txouts": 101,
  "bogosize": 7272,
  "hash_serialized_3": "eecf590f58b8b54368aef83d57c1df658a860e3394406deccbb86639343fd3b6",
  "total_amount": 5050.00000000,
  "transactions": 101,
  "disk_size": 0
}
PS C:\Users\szatm\Desktop\Blockchain-Lab01>
```

Part B - Constructing the raw transaction

The manually chosen fee is the difference between inputs and outputs

Input: 50.00000000 BTC

Destination: 10.00000000 BTC

Change: 39.99990000 BTC

Fee: 50 - 10 - 39.9999 = 0.00010000 BTC (10,000 satoshis)

I created and signed the raw transaction with one input and two P2WPKH outputs. Bitcoin Core decoded it as version 2, size 222 bytes, weight 561, and vsize 141 vB.

```
PS C:\Users\szatm\Desktop\Blockchain-Lab01> docker compose exec -T bitcoind bitcoin-cli decoderawtransaction $SIGNED_HEX
{
  "txid": "2a6d10cdfab21133f58d25022e58b1bcdb65a5a73626e963bdd5596f35d21407",
  "hash": "a70b1819be969bc49cd59ca00a9dab7cd16f288f2bc56cde6506f18c74cf0a8a",
  "version": 2,
  "size": 222,
  "vsize": 141,
  "weight": 561,
  "locktime": 0,
  "vin": [
    {
      "txid": "b64d9ac2ec91ed2aa71738f2cdb689768b2f53d04f61d7eef3528b2d056fde38",
      "vout": 0,
      "scriptSig": {
        "asm": "",
        "hex": ""
      },
      "txinwitness": [
        "3044022075b93fabb1f1145738871711a38cde66ff521c39f721ce11212c1b54f19c3853022065ac688462175aa7c7dd60ddde057c5ed863102c516950999d6ed606e2e4509001",
        "03c619ff767c4df9f5a16ccb368ae23a8bde75ff4a9a99cf20e102fd682b962b14"
      ],
      "sequence": 4294967293
    }
  ],
  "vout": [
    {
      "value": 10.00000000,
      "n": 0,
      "scriptPubKey": {
        "asm": "0 483b7342133ce93eb1740ea74d960c0b016092b0",
        "desc": "addr(bcrt1qfqahxssn8n5navt5p6n5m9svpvqkpy4spk3v90)#03hnt4nm",
        "hex": "0014483b7342133ce93eb1740ea74d960c0b016092b0",
        "address": "bcrt1qfqahxssn8n5navt5p6n5m9svpvqkpy4spk3v90",
        "type": "witness_v0_keyhash"
      }
    },
    {
      "value": 39.99990000,
      "n": 1,
      "scriptPubKey": {
        "asm": "0 4370cfbcba1d4d2704b03f2d605ac586d528c8a3",
        "desc": "addr(bcrt1qgdcvl096r4xjwp9s8ukkqkk9sm2j3j9ry33ymd)#jkemm64w",
        "hex": "00144370cfbcba1d4d2704b03f2d605ac586d528c8a3",
        "address": "bcrt1qgdcvl096r4xjwp9s8ukkqkk9sm2j3j9ry33ymd",
        "type": "witness_v0_keyhash"
      }
    }
  ]
}
PS C:\Users\szatm\Desktop\Blockchain-Lab01>
```

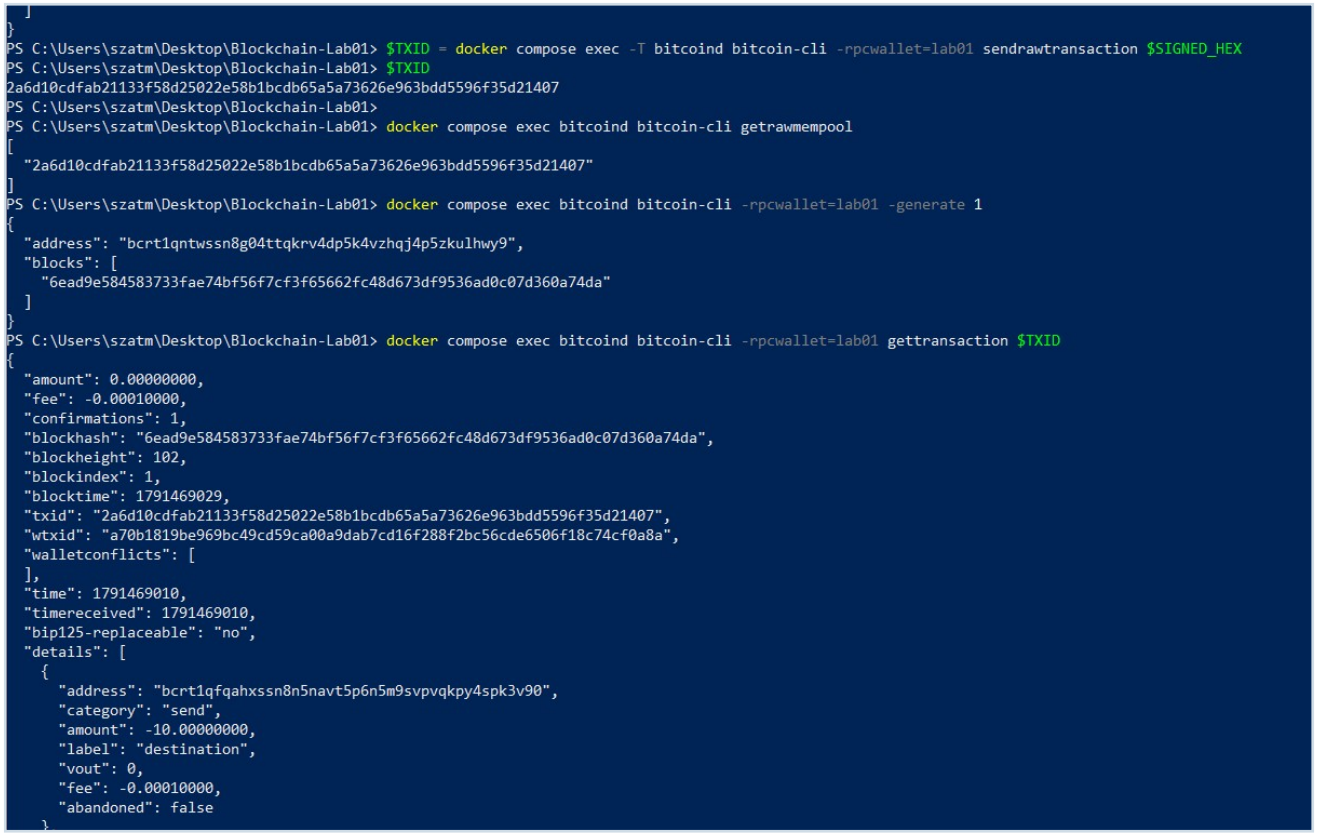
Part B - Confirmation and actual fee

Bitcoin Core wallet: fee -0.00010000 BTC and one confirmation

I broadcast the signed transaction, observed its txid in the mempool, and mined block 102. The gettransaction response reported fee -0.00010000 BTC (negative from my wallet's point of view) and confirmations 1.

txid: 2a6d10cdfab21133f58d25022e58b1bcdb65a5a73626e963bdd5596f35d21407

Effective fee rate: 10,000 satoshis / 141 vB = approximately 70.92 sat/vB.

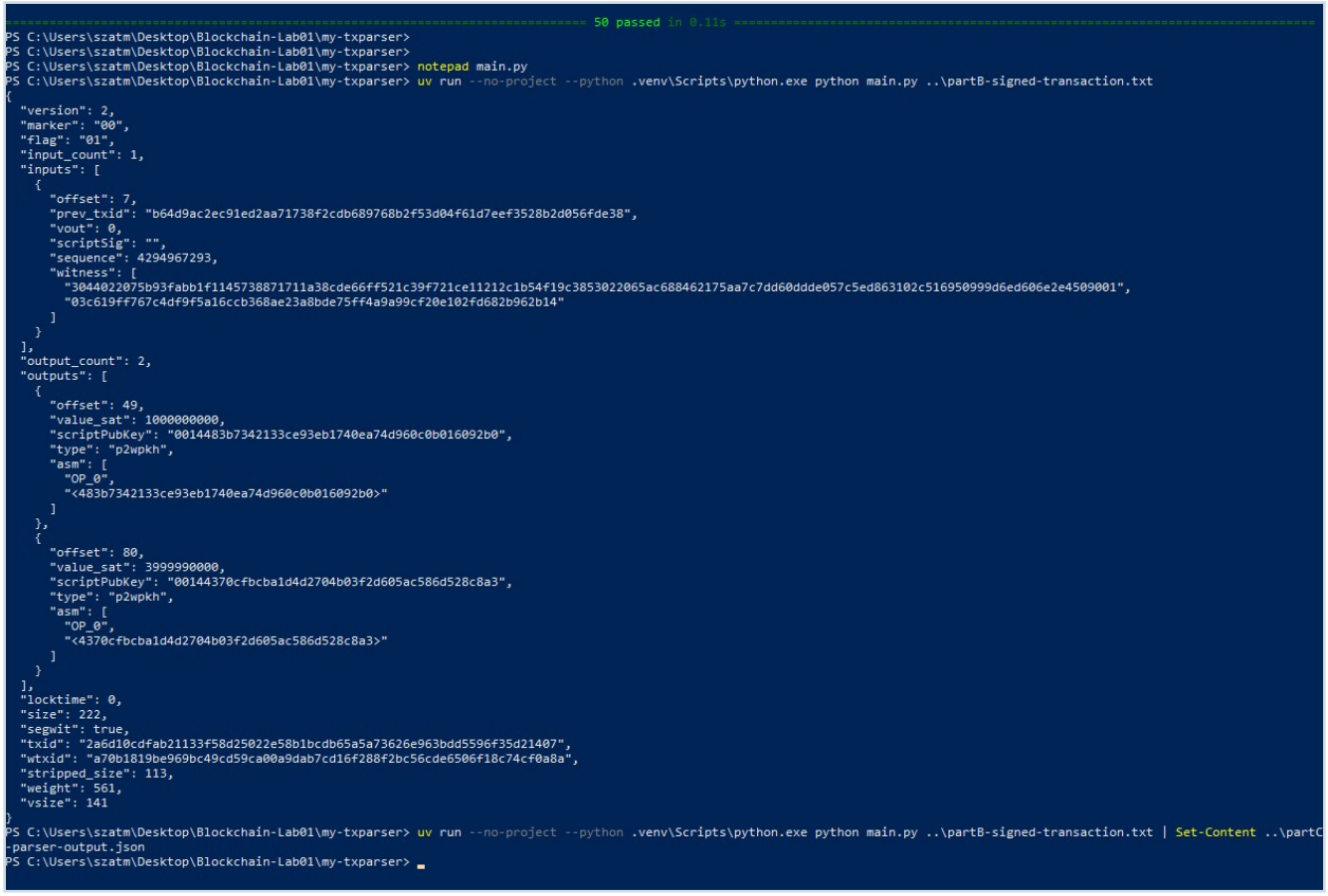


Part C - My own transaction parser

Raw SegWit transaction parsed from my Part B signed hexadecimal data

I implemented the parser in Python and used it to read the Part B transaction. The parser reports byte offsets, the witness stack, inputs, outputs, both hashes, script types, weight and virtual size.

The Part B transaction was detected as SegWit with marker 00, flag 01, one input starting at byte offset 7 and two outputs: 1,000,000,000 and 3,999,990,000 satoshis.

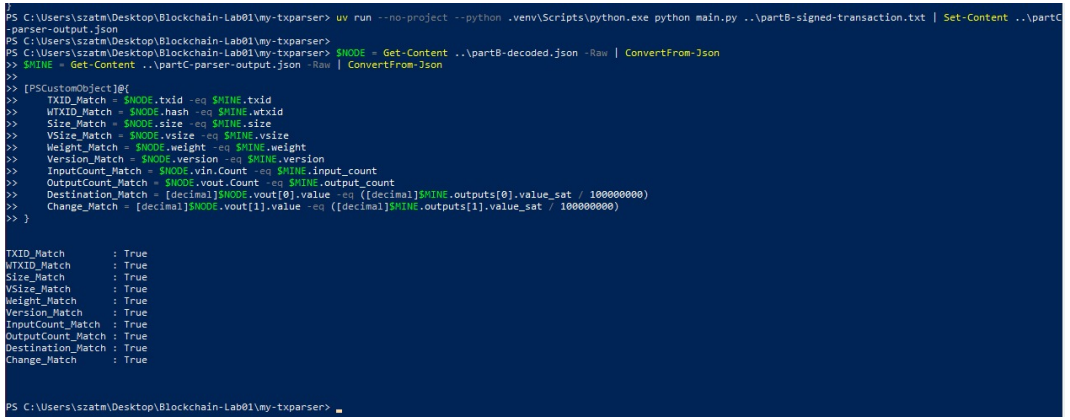


Part C - Verifying my parser

Direct Bitcoin Core comparison and automated test results

I compared the parser JSON with the decoderawtransaction output. All 10 checked fields returned True: txid, wtxid, size, vsize, weight, version, input count, output count and both output amounts.

Bitcoin Core calls the witness-inclusive transaction hash "hash"; my parser calls it "wtxid". Bitcoin Core displays output values in BTC, while my parser uses satoshis.




```
test_parser.py::test_output_classification[6a0548656c6c6f-nulldata] PASSED                                                
```

Part D - Testing the high-fee protection

One 50 BTC input, one 10 BTC output, no change: an implicit 40 BTC fee

I deliberately left out the change output. The first sendrawtransaction attempt was rejected with error -25 because the fee exceeded the default RPC safety limit. This is a safety rule, not a consensus rule.

```
PS C:\Users\szatm\Desktop\Blockchain-Lab01>
PS C:\Users\szatm\Desktop\Blockchain-Lab01> docker compose exec -T bitcoind bitcoin-cli -rpcwallet=lab01 sendrawtransaction $SIGNED_HEX_D
error code: -25
error message:
Fee exceeds maximum configured by user (e.g. -maxtxfee, maxfeerate)

PS C:\Users\szatm\Desktop\Blockchain-Lab01> docker compose exec -T bitcoind bitcoin-cli -rpcwallet=lab01 sendrawtransaction $SIGNED_HEX_D
error code: -25
error message:
Fee exceeds maximum configured by user (e.g. -maxtxfee, maxfeerate)

PS C:\Users\szatm\Desktop\Blockchain-Lab01>
```

I then used maxfeerate=0 on regtest only. Bitcoin Core accepted the transaction into the mempool.

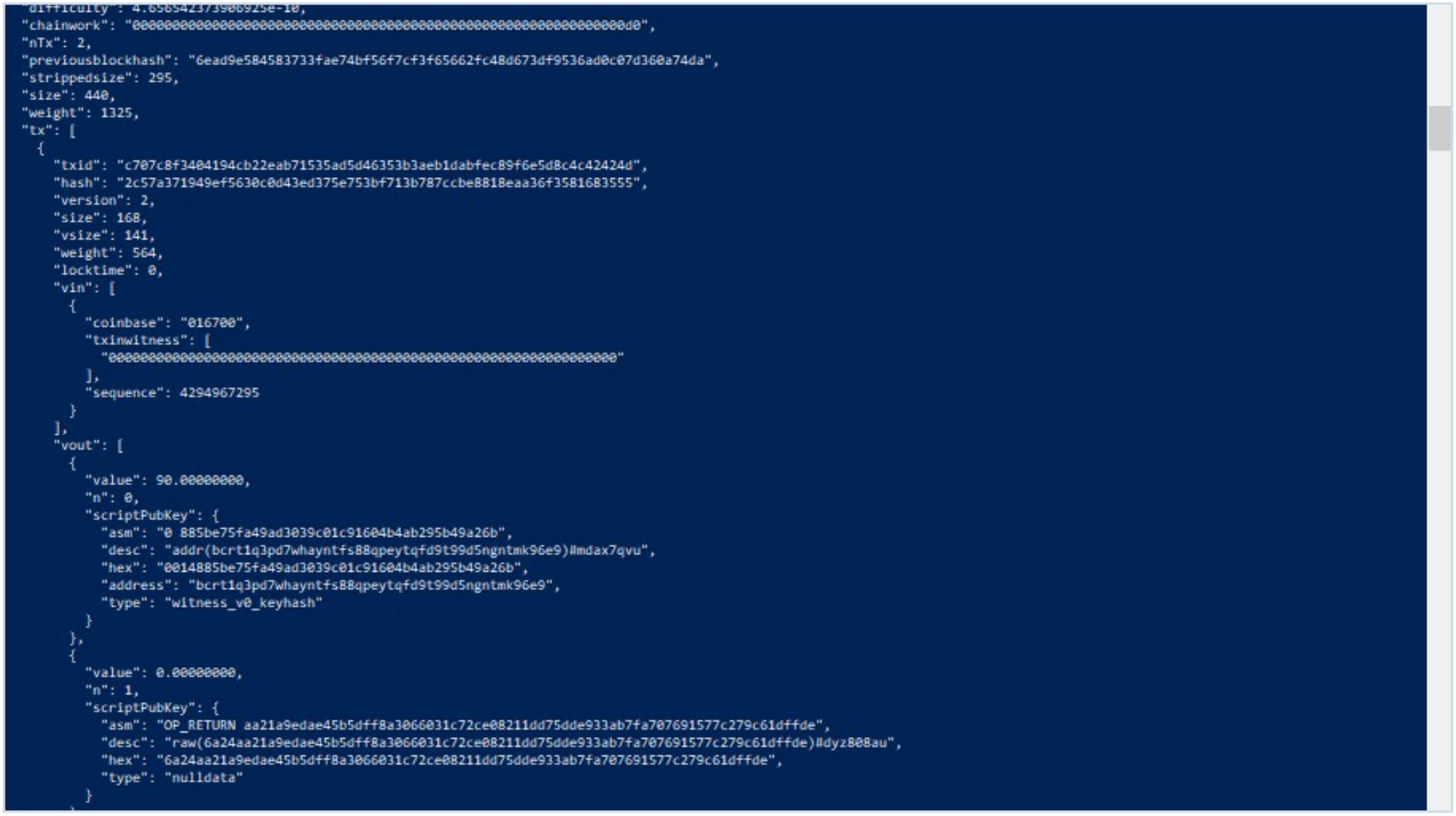
```
PS C:\Users\szatm\Desktop\Blockchain-Lab01>
PS C:\Users\szatm\Desktop\Blockchain-Lab01>
PS C:\Users\szatm\Desktop\Blockchain-Lab01> $TXID_D = docker compose exec -T bitcoind bitcoin-cli -rpcwallet=lab01 sendrawtransaction $SIGNED_HEX_D 0
PS C:\Users\szatm\Desktop\Blockchain-Lab01> $TXID_D
d337721793ecc854b3c019f83a8027edd0a5ac9d88fe70ac8e0cb91ba104e319
PS C:\Users\szatm\Desktop\Blockchain-Lab01> docker compose exec bitcoind bitcoin-cli getrawmempool
[
  "d337721793ecc854b3c019f83a8027edd0a5ac9d88fe70ac8e0cb91ba104e319"
]
PS C:\Users\szatm\Desktop\Blockchain-Lab01> docker compose exec -T bitcoind bitcoin-cli getrawmempool | Set-Content ".\partD-mempool.json"
PS C:\Users\szatm\Desktop\Blockchain-Lab01>
```

Part D - Block 103 and miner reward

The block paid 90 BTC: 50 BTC subsidy plus my 40 BTC fee

After mining block 103, getblock ... 2 showed two transactions. The coinbase transaction (tx[0]) had an output of 90 BTC. My transaction (tx[1]) had one 10 BTC output and a 40 BTC fee.

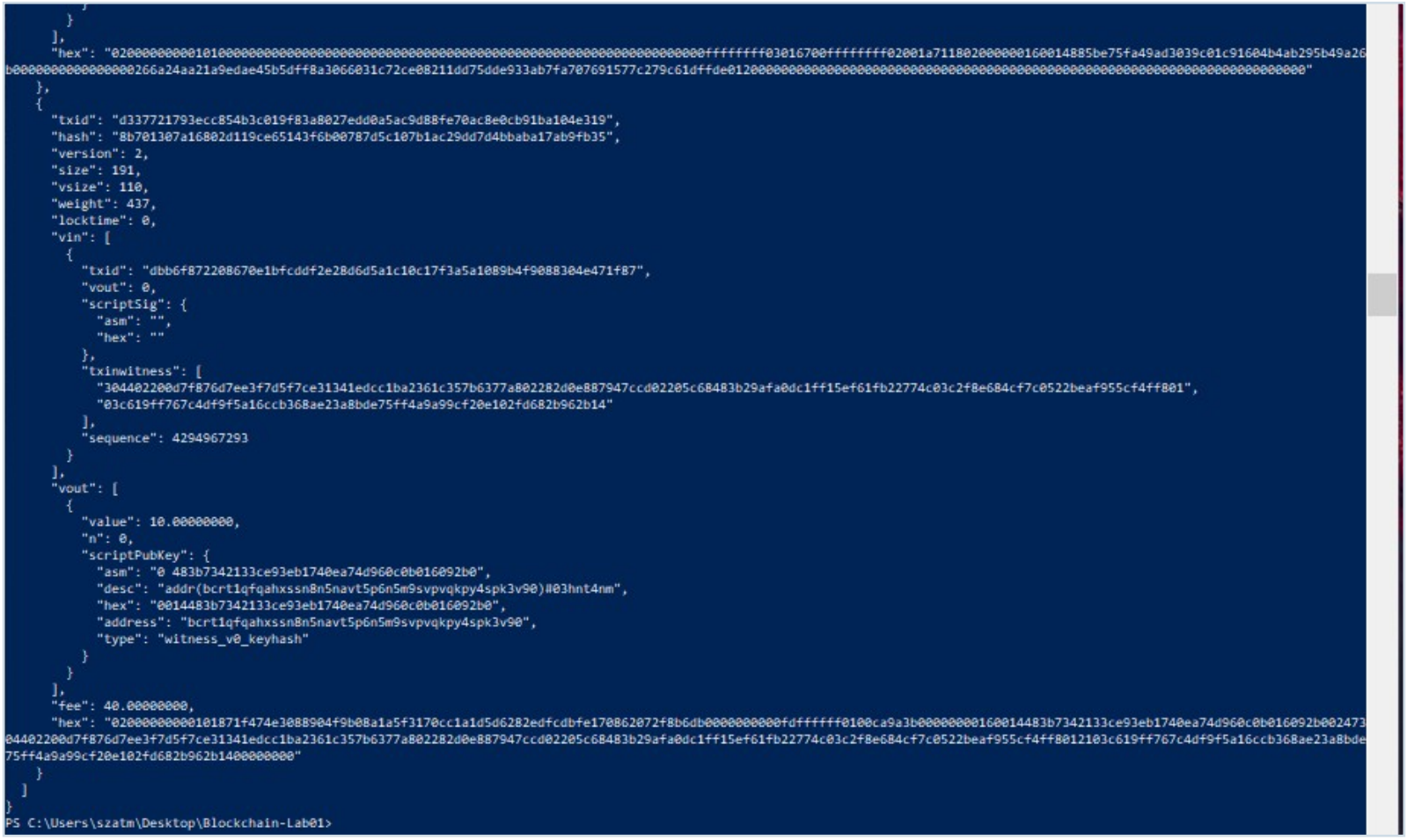
The consensus rules accept this: output value does not exceed input value, and the miner may claim the subsidy plus transaction fees. Since I mined the block, the 40 BTC fee returned to my wallet as part of the immature coinbase.



Part D - Inspecting the included payment

The decoded transaction has a single 10 BTC output and fee 40 BTC

The second decoded transaction in block 103 confirms the missing change output. Unlike the Part B transaction, it has only one output, valued at 10 BTC. The missing remainder is a fee to the miner.



Why Bitcoin has no explicit fee field

Required short explanation - fewer than 300 words

Bitcoin does not need a separate transaction fee field because the value of every input is already known from the previous output it spends. The fee is simply the sum of the input values minus the sum of the new output values. Adding another fee field would repeat information that can already be calculated and could even create disagreements if the two amounts were different.

This design makes checking a transaction simpler. A node checks that the outputs are not worth more than the inputs, and the leftover amount becomes the fee. Miners can collect the block subsidy together with the transaction fees. The sender decides how much to leave as a fee, which allows a fee market to form without Bitcoin setting one fixed transaction price. Leaving out an additional field also saves some space.

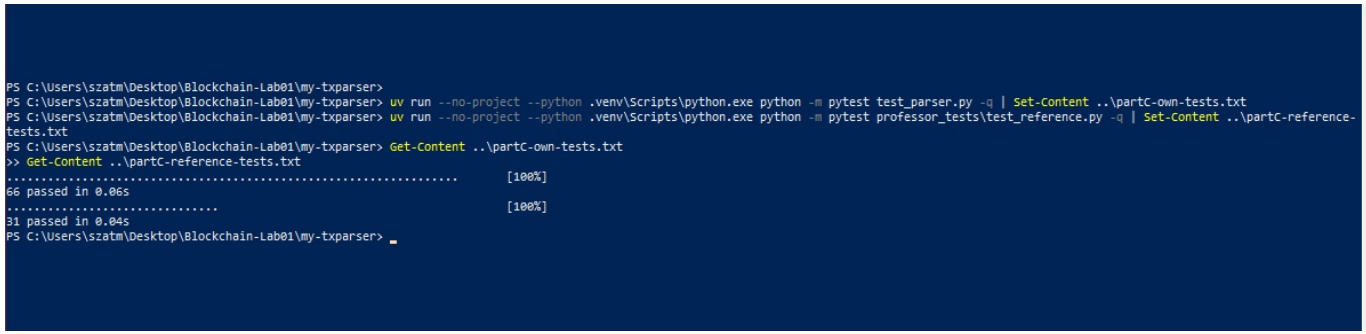
The disadvantage is that making a mistake with the change output can create a very large fee. I saw this in Part D when I spent a 50 BTC input but only created a 10 BTC output, producing a 40 BTC fee. It also means my parser cannot calculate the fee from the raw transaction alone without looking up the values of the spent outputs. Wallets and Bitcoin Core therefore have safety checks, such as maxfeerate, to help prevent this mistake.

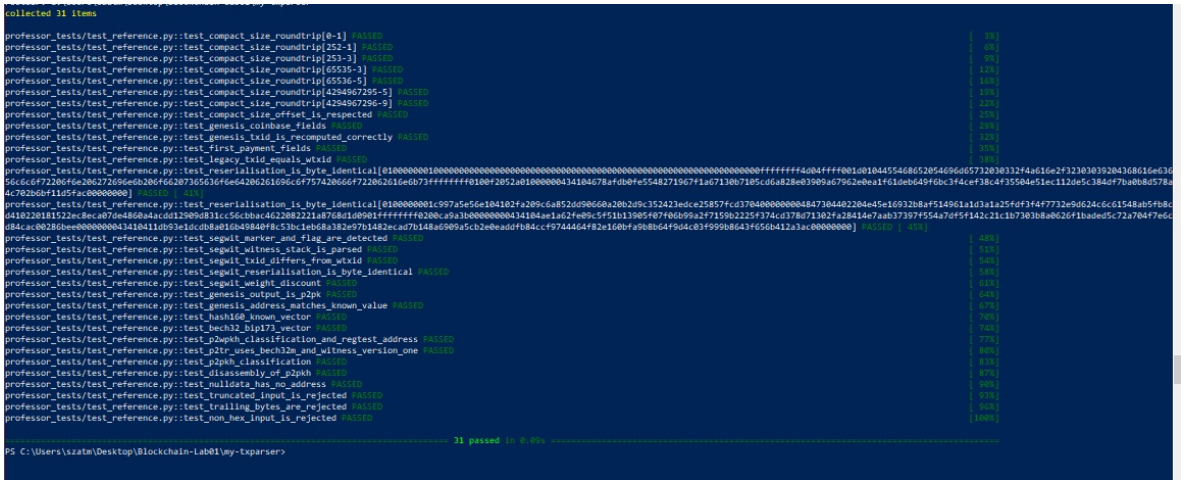
Part C - Final automated verification

All 66 local tests and all 31 provided reference tests passed

After adding Bitcoin address generation and adapting my parser to the expected package interface, I reran both test suites. My own tests finished with 66 passed. The professor-provided reference suite finished with 31 passed, without modifying those tests.

I also tested mainnet and regtest output addresses. The genesis P2PK public key gave the known mainnet address, while the two P2WPKH outputs from Part B produced their expected bcrt1q regtest addresses. An OP_RETURN output correctly has no address.





Submission and verification summary

Source code is available in a separate public repository

Parts A and B: block height 101 and 102, UTXO evidence, confirmed 10 BTC transaction and the measured 0.0001 BTC fee.

Part C: independent parser; 66/66 local tests and 31/31 professor reference tests passed; ten Bitcoin Core comparisons matched.

Part D: first broadcast rejected with error -25; override accepted; block 103 recorded a 90 BTC coinbase output.

Repository

https://github.com/korneliasara/blockchain-lab01-bitcoin-parser

Verification: My 66 local tests and all 31 professor-provided reference tests passed in separate runs. The reference tests were run unchanged. I also verified output-address generation for the historical genesis output and both regtest P2WPKH outputs. Source code with these changes was pushed to the public GitHub repository.

This report uses screenshots from the terminal sessions and describes what I did in my own words. All activity took place on a private regtest network.